



# Signals and Systems

## 实验报告

实验名称 \_\_\_\_\_ Lecture4

实验地点 \_\_\_\_\_

姓 名 \_\_\_\_\_ 金宣妤

学 号 \_\_\_\_\_ 3240100333

实验日期 \_\_\_\_\_ 2026.5.26

指导老师 \_\_\_\_\_ Prof. Xiaoming Chen

# Contents

|          |                                                                        |           |
|----------|------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Experimental Objectives</b>                                         | <b>1</b>  |
| <b>2</b> | <b>Experimental Principles</b>                                         | <b>1</b>  |
| 2.1      | Discrete-Time Convolution . . . . .                                    | 1         |
| 2.2      | Image Filtering with 1D Filters . . . . .                              | 1         |
| 2.3      | Discrete-Time Fourier Series (DTFS) . . . . .                          | 2         |
| <b>3</b> | <b>Experimental Contents, Results, and Analysis</b>                    | <b>2</b>  |
| 3.1      | Problem 1: Discrete-Time Convolution . . . . .                         | 2         |
| 3.2      | Problem 2: Image Processing with 1D Filters . . . . .                  | 6         |
| 3.3      | Problem 3: Fourier Series Representation of Periodic Signals . . . . . | 13        |
| <b>4</b> | <b>Experimental Conclusions</b>                                        | <b>20</b> |

# 1 Experimental Objectives

- Understand how discrete-time convolution works and learn how to use MATLAB's `conv` function, including how to track the correct time indices manually.
- Learn the overlap-add method as a way to compute convolution in blocks, and check that it gives the same result as direct convolution.
- Apply 1D lowpass filters to a 2D image using a separable filter structure. Compare Butterworth (IIR) and Remez (FIR) filters in terms of magnitude response, phase response, and image distortion.
- Use the Discrete-Time Fourier Series (DTFS) to analyze and synthesize periodic discrete-time signals. Understand the relationship between time-domain signals and their frequency-domain coefficients.

# 2 Experimental Principles

## 2.1 Discrete-Time Convolution

The convolution of two discrete-time sequences  $h[n]$  and  $x[n]$  is defined as:

$$y[n] = \sum_{m=-\infty}^{\infty} h[m] x[n-m] \quad (1)$$

MATLAB's `conv(h,x)` computes this sum for finite-length sequences. If  $x[n]$  is nonzero only on  $[n_x, n_x + N_x - 1]$  and  $h[n]$  is nonzero only on  $[n_h, n_h + N_h - 1]$ , then  $y[n]$  is nonzero only on:

$$(n_x + n_h) \leq n \leq (n_x + n_h) + N_x + N_h - 2 \quad (2)$$

Because `conv` does not track time indices automatically, we must compute `ny` by hand using this formula.

## 2.2 Image Filtering with 1D Filters

A 2D image  $x[m, n]$  can be filtered using a separable 2D filter: first filter every column of  $x$  to get an intermediate matrix  $z$ , then filter every row of  $z$  to get the output  $y$ . This is equivalent to applying a 2D filter, but much simpler to implement with 1D tools.

For noncausal filters (needed so that the image is not shifted), a causal filter of order  $n$  introduces a delay of  $d = n/2$  samples. We compensate by appending  $d$  zeros to the

input before filtering, then taking the last  $N$  samples.

## 2.3 Discrete-Time Fourier Series (DTFS)

For a periodic signal  $x[n]$  with fundamental period  $N$ , the DTFS pair is:

$$x[n] = \sum_{k=0}^{N-1} a_k e^{jk(2\pi/N)n} \quad (\text{synthesis}) \quad (3)$$

$$a_k = \frac{1}{N} \sum_{n=0}^{N-1} x[n] e^{-jk(2\pi/N)n} \quad (\text{analysis}) \quad (4)$$

In MATLAB:  $\mathbf{a} = (1/N)*\text{fft}(\mathbf{x})$  and  $\mathbf{x} = N*\text{ifft}(\mathbf{a})$ . A real-valued  $x[n]$  requires its DTFS coefficients to be conjugate symmetric, i.e.  $a_k = a_{-k}^*$ .

## 3 Experimental Contents, Results, and Analysis

### 3.1 Problem 1: Discrete-Time Convolution

#### (a) Basic convolution with time index tracking

Given  $h[n] = 2\delta[n+1] - 2\delta[n-1]$  and  $x[n] = \delta[n] + \delta[n-2]$ , we compute  $y[n] = x[n] * h[n]$  and find the correct time indices for the output.

code:

```
1 clear;
2 clc;
3
4 nh=-1:1;h=[2 0 -2];
5 nx=0:2;x=[1 0 1];
6 ny=-1:3;y=conv(h,x);
7 stem(ny,y);
8 xlabel('n'); ylabel('y[n]');
```

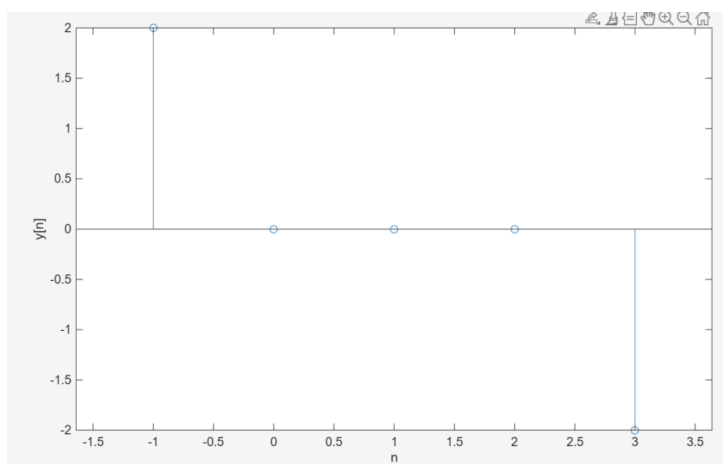


Figure 1: 1(a): Convolution output  $y[n]$  with correct time indices.

**Result:**  $y[n]$  is nonzero on  $[-1, 3]$ , with values  $[2, -2, 2, -2]$  at  $n = -1, 1, 1, 3$  (after superposition). The output length is  $N_x + N_h - 1 = 3 + 3 - 1 = 5$ .

### (b) General formula for time indices

For two finite sequences with index ranges  $\mathbf{nh} = [a:b]$  and  $\mathbf{nx} = [c:d]$ , the output index vector  $\mathbf{ny}$  should be:

$$\mathbf{ny} = (a + c) : (b + d) \quad (5)$$

**Derivation:** Consider  $h[n] = \delta[n - a] + \delta[n - b]$  and  $x[n] = \delta[n - c] + \delta[n - d]$ . Then:

$$y[n] = \delta[n - a - c] + \delta[n - a - d] + \delta[n - b - c] + \delta[n - b - d]$$

So  $y[n]$  starts at  $n = a + c$  and ends at  $n = b + d$ .

**code:**

```
1 % General formula (given nh and nx as vectors of all indices)
2 ny_start = nh(1) + nx(1);
3 ny_end   = nh(end) + nx(end);
4 ny = ny_start : ny_end;
```

### (c) Truncated infinite sequences

Given:

$$x[n] = \left(\frac{1}{2}\right)^{n-2} u[n - 2], \quad h[n] = u[n + 2]$$

Both sequences are infinite, so we truncate:  $x[n]$  for  $0 \leq n \leq 24$  and  $h[n]$  for  $0 \leq n \leq 14$ .

**code:**

```

1 clear;
2 clc;
3
4 nh=0:14; h=ones(1,15);
5
6 nx=0:24; x=(nx>=2).*(0.5.^(nx-2));
7
8 ny=0:38; y=conv(h,x);
9
10 stem(ny,y);
11 xlabel('n'); ylabel('y[n]');

```

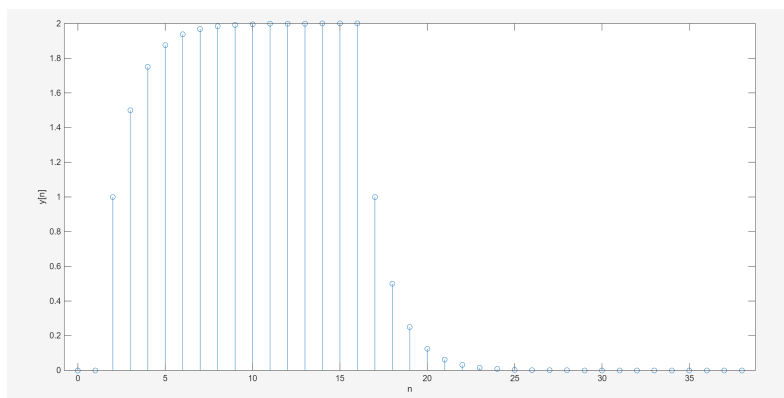


Figure 2: P1(c)

**Analysis:** Since  $h[n]$  was truncated to 15 samples and  $x[n]$  to 25 samples, the output is valid only for  $n \leq 12$ . Beyond that, the convolution window reaches the truncated end of  $h[n]$ , so missing samples are treated as zero, causing errors in the result.

#### (d) Direct convolution

Given  $h[n] = (0.9)^n(u[n] - u[n - 10])$  and  $x[n] = \cos(n^2) \sin(2\pi n/5)$ , compute  $y[n] = h[n] * x[n]$  for  $0 \leq n \leq 99$ . **code:**

```

1 clear;
2 clc;
3 n=0:99;
4 x=cos(n.^2).*sin((2/5)*pi.*n);
5 h=(0.9).^n.*[ones(1,10) zeros(1,90)];
6

```

```

7  y1=conv(h,x);
8  y = y1(1:100); ny = 0:99;
9
10 stem(ny,y);
11 xlabel('n'); ylabel('y[n]');

```

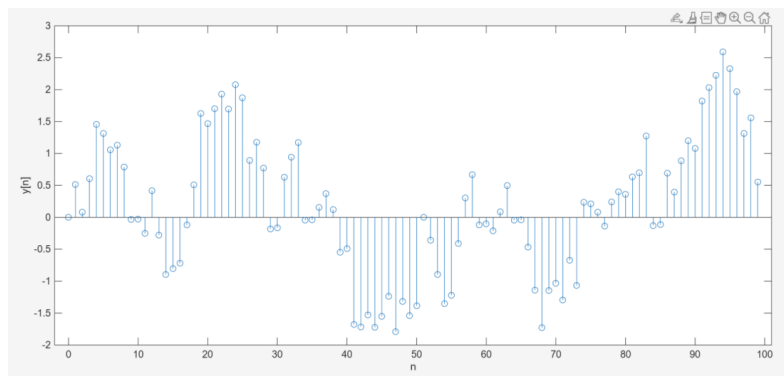


Figure 3: P1(d):  $y[n] = h[n] * x[n]$  computed by direct convolution.

### (e) Overlap-add method

We split  $x[n]$  into two blocks of length  $L = 50$  and convolve each with  $h[n]$ , then overlap-add the results. The shift  $k = L = 50$  because  $x_1[n]$  starts at  $n = 50$ . **code:**

```

1  clear;
2  clc;
3  n=0:99;
4  x=cos(n.^2).*sin((2/5)*pi.*n);
5  h=(0.9).^n.*[ones(1,10) zeros(1,90)];
6
7  ny=0:198; y=conv(h,x);
8
9  nx0=0:49; x0=cos(nx0.^2).*sin((2/5)*pi.*nx0);
10 nx1=nx0+50; x1=cos(nx1.^2).*sin((2/5)*pi.*nx1);
11 y0=conv(h,x0);
12 y1=conv(h,x1);
13 y_final=[y0 zeros(1,50)]+[zeros(1,50) y1];
14
15 stem(n,y(1:100),'r');
16 hold on
17 stem(n,y_final(1:100));

```

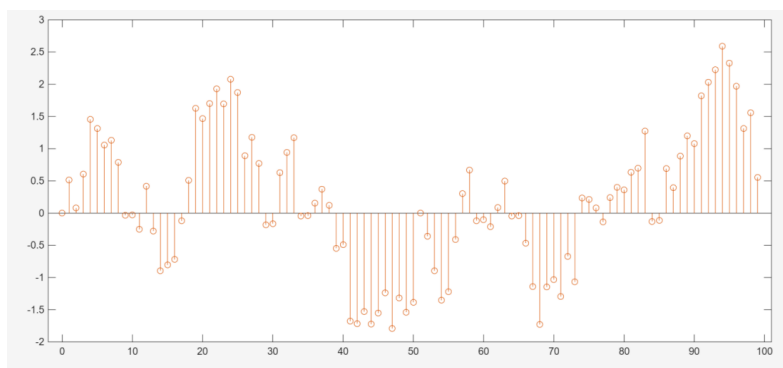


Figure 4: P1(e): Overlap-add result matches direct convolution.

**Conclusion:** The two methods give the same result (difference  $< 10^{-12}$ ), which confirms the linearity and time-invariance of convolution.

## 3.2 Problem 2: Image Processing with 1D Filters

### (a) Creating three lowpass filters

code:

```

1 clear; clc;
2 load plus.mat
3 colormap(gray); image(64*x);
4 title('Original image');
5
6 wc = 0.4;
7 n1 = 10; n2 = 4; n3 = 12;
8
9 [b1,a1] = butter(n1, wc);
10 a2 = 1; b2 = remez(n2, [0 wc-0.04 wc+0.04 1], [1 1 0 0]);
11 a3 = 1; b3 = remez(n3, [0 wc-0.04 wc+0.04 1], [1 1 0 0]);

```

### (b) Frequency response (magnitude and phase)

code:

```

1 clear; clc;
2 load plus.mat
3
4 % 原始图像
5 % figure;

```

```
6 % colormap(gray); image(64*x);
7 % title('原始图像');
8
9 wc = 0.4;
10 n1 = 10; n2 = 4; n3 = 12;
11 [b1,a1] = butter(n1, wc);
12 a2 = 1; b2 = remez(n2, [0 wc-0.04 wc+0.04 1], [1 1 0 0]);
13 a3 = 1; b3 = remez(n3, [0 wc-0.04 wc+0.04 1], [1 1 0 0]);
14
15 figure;
16 [H1,w1] = freqz(b1,a1,512);
17 [H2,w2] = freqz(b2,a2,512);
18 [H3,w3] = freqz(b3,a3,512);
19
20 subplot(3,2,1); plot(w1/pi, abs(H1));
21 xlabel('\omega/\pi'); ylabel('|H|'); title('Filter1(Butter) 幅频'); grid
    on;
22 subplot(3,2,2); plot(w1/pi, angle(H1));
23 xlabel('\omega/\pi'); ylabel('Phase'); title('Filter1(Butter) 相频');
    grid on;
24
25 subplot(3,2,3); plot(w2/pi, abs(H2));
26 xlabel('\omega/\pi'); ylabel('|H|'); title('Filter2(Remez n=4) 幅频');
    grid on;
27 subplot(3,2,4); plot(w2/pi, angle(H2));
28 xlabel('\omega/\pi'); ylabel('Phase'); title('Filter2(Remez n=4) 相频');
    grid on;
29
30 subplot(3,2,5); plot(w3/pi, abs(H3));
31 xlabel('\omega/\pi'); ylabel('|H|'); title('Filter3(Remez n=12) 幅频');
    grid on;
32 subplot(3,2,6); plot(w3/pi, angle(H3));
33 xlabel('\omega/\pi'); ylabel('Phase'); title('Filter3(Remez n=12) 相频')
    ; grid on;
```

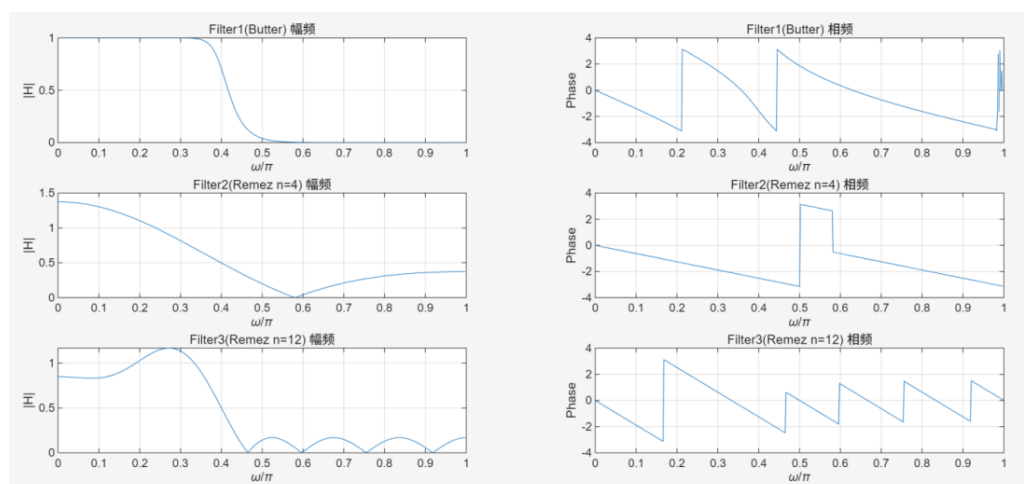


Figure 5: P2(b): Magnitude and phase responses of the three filters. Filter 2 and Filter 3 (FIR) have linear phase; Filter 1 (IIR) does not.

**Linear phase:** Filter 2 and Filter 3 are FIR filters designed with `remez`, so their phase is linear. Filter 1 is an IIR Butterworth filter with nonlinear phase.

### (c) Step response

code:

```

1  clear; clc;
2  load plus.mat
3
4  % 原始图像
5  % figure;
6  % colormap(gray); image(64*x);
7  % title('原始图像');
8
9  wc = 0.4;
10 n1 = 10; n2 = 4; n3 = 12;
11 [b1,a1] = butter(n1, wc);
12 a2 = 1; b2 = remez(n2, [0 wc-0.04 wc+0.04 1], [1 1 0 0]);
13 a3 = 1; b3 = remez(n3, [0 wc-0.04 wc+0.04 1], [1 1 0 0]);
14
15 u = [1, zeros(1,20)];
16 u_step = ones(1,21);
17 n_step = 0:20;
18
19 figure;
```

```

20 s1 = filter(b1,a1,u_step);
21 s2 = filter(b2,a2,u_step);
22 s3 = filter(b3,a3,u_step);
23
24 subplot(3,1,1); stem(n_step,s1); title('Filter1 阶跃响应'); grid on;
25 subplot(3,1,2); stem(n_step,s2); title('Filter2 阶跃响应'); grid on;
26 subplot(3,1,3); stem(n_step,s3); title('Filter3 阶跃响应'); grid on;

```

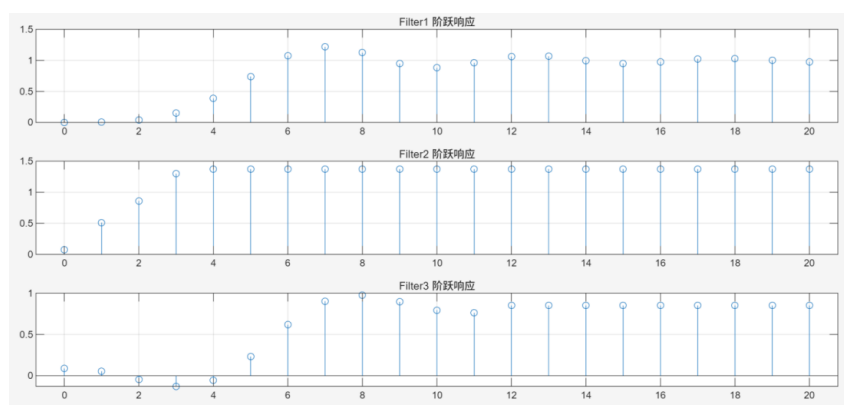


Figure 6: P2(c): Step responses of the three filters. Filter 3 has the largest overshoot.

**Largest overshoot:** Filter 3 (Remez, order 12) has the steepest transition band, which causes the largest ringing in its step response. This ringing will appear as artifacts around sharp edges in filtered images.

#### (d) Noncausal filtering of a single image column

We take column 16 of the image and apply Filter 1 noncausally. The delay  $d = n_1/2 = 5$  is removed by appending zeros and shifting the output. **code:**

```

1 clear; clc;
2 load plus.mat
3
4 % 原始图像
5 % figure;
6 % colormap(gray); image(64*x);
7 % title('原始图像');
8
9 wc = 0.4;
10 n1 = 10; n2 = 4; n3 = 12;
11 [b1,a1] = butter(n1, wc);

```

```

12 a2 = 1; b2 = remez(n2, [0 wc-0.04 wc+0.04 1], [1 1 0 0]);
13 a3 = 1; b3 = remez(n3, [0 wc-0.04 wc+0.04 1], [1 1 0 0]);
14 x16 = x(:,16);
15
16 y16_f1 = filter(b1,a1,[x16; zeros(n1/2,1)]);
17 y16_f1 = y16_f1(n1/2+1:end);
18
19 figure;
20 plot(x16,'b'); hold on; plot(y16_f1,'r');
21 legend('原始x16','滤波后y16');
22 title('Filter1对第16列的响应'); grid on;

```

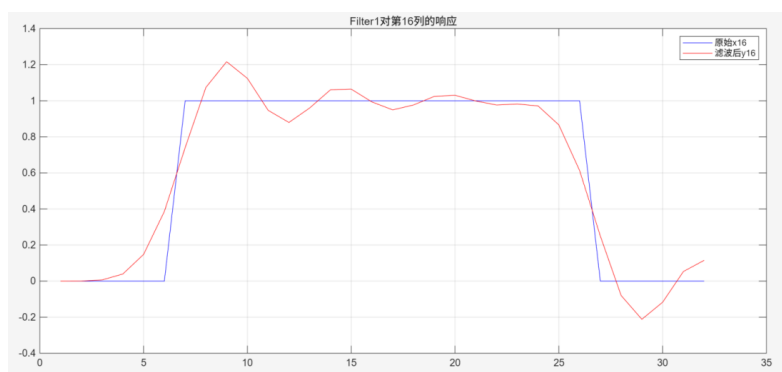


Figure 7: P2(d): Column 16 before and after noncausal filtering with Filter 1. The discontinuities in  $x_{16}$  line up with the smoothed versions in  $y_{16}$ .

### (e) Filters 2 and 3 on column 16

code:

```

1 clear; clc;
2 load plus.mat
3
4 % 原始图像
5 % figure;
6 % colormap(gray); image(64*x);
7 % title('原始图像');
8
9 wc = 0.4;
10 n1 = 10; n2 = 4; n3 = 12;
11 [b1,a1] = butter(n1, wc);
12 a2 = 1; b2 = remez(n2, [0 wc-0.04 wc+0.04 1], [1 1 0 0]);

```

```

13 a3 = 1; b3 = remez(n3, [0 wc-0.04 wc+0.04 1], [1 1 0 0]);
14 x16 = x(:,16);
15
16 y16_f2 = filter(b2,a2,[x16; zeros(n2/2,1)]);
17 y16_f2 = y16_f2(n2/2+1:end);
18
19 y16_f3 = filter(b3,a3,[x16; zeros(n3/2,1)]);
20 y16_f3 = y16_f3(n3/2+1:end);
21
22 figure;
23 subplot(3,1,1); plot(x16,'b'); title('原始x16'); grid on;
24 subplot(3,1,2); plot(y16_f2,'r'); title('Filter2输出'); grid on;
25 subplot(3,1,3); plot(y16_f3,'r'); title('Filter3输出'); grid on;

```

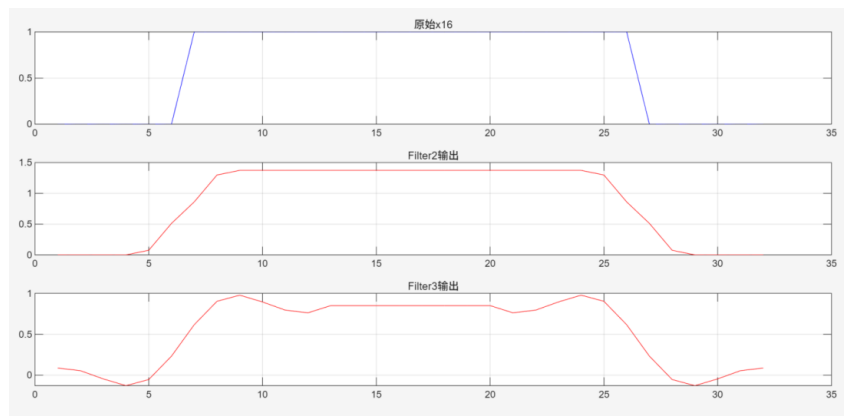


Figure 8: P2(e): Column 16 filtered by Filter 2 (middle) and Filter 3 (bottom). No delay or advance is present when the delay is correctly compensated.

#### (f) `filt2d.m` function

The function `filt2d(b, a, d, x)` filters every column of the image, stores the result in  $z$ , then filters every row of  $z$ . Each column/row is zero-padded by  $d$  samples before filtering, and the last  $N$  samples are taken to remove the delay.

#### (g) Display filtered images

code:

```

1 clear; clc;
2 load plus.mat
3

```

```

4 % 原始图像
5 % figure;
6 % colormap(gray); image(64*x);
7 % title('原始图像');
8
9 wc = 0.4;
10 n1 = 10; n2 = 4; n3 = 12;
11 [b1,a1] = butter(n1, wc);
12 a2 = 1; b2 = remez(n2, [0 wc-0.04 wc+0.04 1], [1 1 0 0]);
13 a3 = 1; b3 = remez(n3, [0 wc-0.04 wc+0.04 1], [1 1 0 0]);
14
15 y1 = filt2d(b1, a1, n1/2, x);
16 y2 = filt2d(b2, a2, n2/2, x);
17 y3 = filt2d(b3, a3, n3/2, x);
18
19 figure;
20 colormap(gray);
21 subplot(1,4,1); image(64*x); title('原始图像');
22 subplot(1,4,2); image(64*y1); title('Filter1(Butter)');
23 subplot(1,4,3); image(64*y2); title('Filter2(Remez 4)');
24 subplot(1,4,4); image(64*y3); title('Filter3(Remez 12)');

```

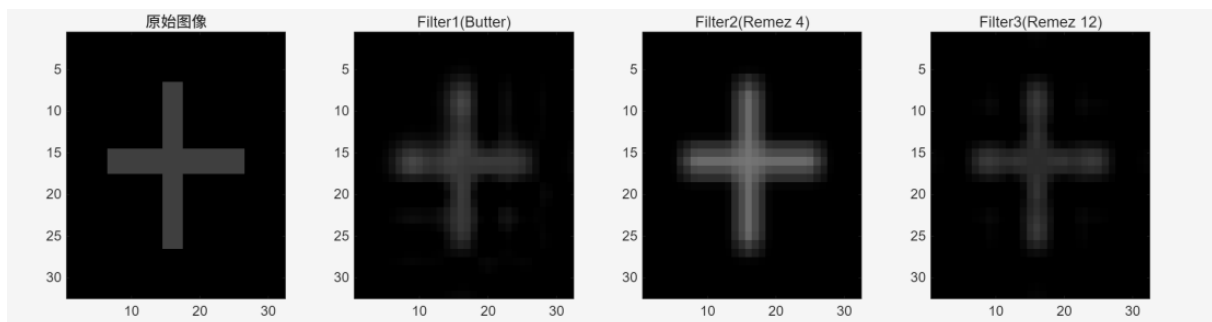


Figure 9: P2(g): Original image and the three lowpass-filtered versions. The “plus” sign in the filtered images should stay centered.

#### (h) Which filter causes more distortion?

**Filter 1 (Butterworth)** causes the most shape distortion. Because it is an IIR filter, its phase response is nonlinear. Different frequency components are delayed by different amounts, which breaks the symmetry of the original image. Filter 2 and Filter 3 are FIR filters with linear phase, so they only blur the image without distorting its shape.

### 3.3 Problem 3: Fourier Series Representation of Periodic Signals

#### Basic Problems

(a) Will  $x[n]$  be real, purely imaginary, or complex?

A periodic discrete-time signal  $x[n]$  is real-valued if and only if its DTFS coefficients satisfy conjugate symmetry:  $a_k = a_{-k}^*$ .

Checking the given coefficients ( $N = 5$ ):

- $a_2 = e^{j\pi/4}$ , and  $a_{-2} = a_2^* = e^{-j\pi/4}$  ✓
- $a_4 = 2e^{j\pi/3}$ , and  $a_{-4} = a_4^* = 2e^{-j\pi/3}$  ✓
- $a_0 = 1$  is real ✓

Since all coefficients satisfy  $a_k = a_{-k}^*$ , we expect  $x[n]$  to be **purely real**.

(b) Values of  $a_0$  through  $a_4$

With  $N = 5$ , the DTFS vector stores  $a_k$  for  $k = 0, 1, 2, 3, 4$ . Negative indices map as:  $a_{-k} = a_{N-k}$ , so  $a_{-4} = a_1$  and  $a_{-2} = a_3$ .

$$a_0 = 1, \quad a_1 = a_{-4}^* = 2e^{-j\pi/3}, \quad a_2 = e^{j\pi/4}, \quad a_3 = a_{-2}^* = e^{-j\pi/4}, \quad a_4 = 2e^{j\pi/3}$$

(c) Synthesize  $x[n]$  and plot

code:

```

1  clear; clc;
2  N = 5;
3  a = [1, 2*exp(-1j*pi/3), exp(1j*pi/4), exp(-1j*pi/4), 2*exp(1j*pi/3)];
4
5  n = 0:N-1;
6  x = zeros(1,N);
7  for k = 0:N-1
8      x = x + a(k+1)*exp(1j*k*(2*pi/N)*n);
9  end
10 x = real(x);

```

```

11
12 figure;
13 stem(n, x);
14 xlabel('n'); ylabel('x[n]');
15 title('Problem3 Basic: 合成信号 x[n], N=5');
16 fprintf('x = %.4f %.4f %.4f %.4f %.4f\n', x(1),x(2),x(3),x(4),x(5));

```

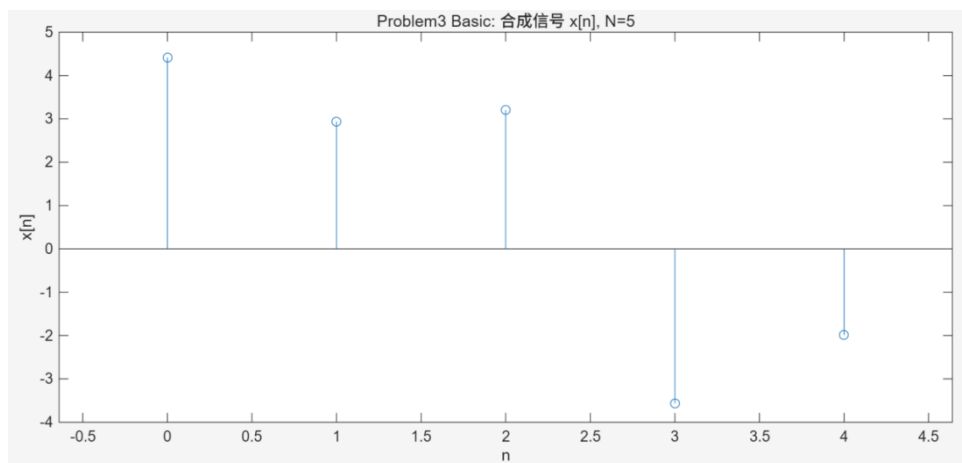


Figure 10: P3(c): One period of the synthesized signal  $x[n]$ . The signal is purely real, confirming the prediction in (a).

#### Numerical values:

$$x[0] \approx 4.4142, \quad x[1] \approx 2.9372, \quad x[2] \approx 3.2001, \quad x[3] \approx -3.5622, \quad x[4] \approx -1.9894$$

#### Intermediate Problems

(d) Define three square waves and plot for  $0 \leq n \leq 63$

code:

```

1 clear; clc;
2 % 定义三个方波一个周期
3 x1 = ones(1,8);
4 x2 = [ones(1,8), zeros(1,8)];
5 x3 = [ones(1,8), zeros(1,24)];
6
7 N1=8; N2=16; N3=32;
8
9 figure;

```

```

10 subplot(3,1,1);
11 stem(0:63, repmat(x1,1,N1/N1*8));
12 title('x1[n], N1=8'); xlabel('n'); grid on;
13
14 subplot(3,1,2);
15 stem(0:63, repmat(x2,1,4));
16 title('x2[n], N2=16'); xlabel('n'); grid on;
17
18 subplot(3,1,3);
19 stem(0:63, repmat(x3,1,2));
20 title('x3[n], N3=32'); xlabel('n'); grid on;

```

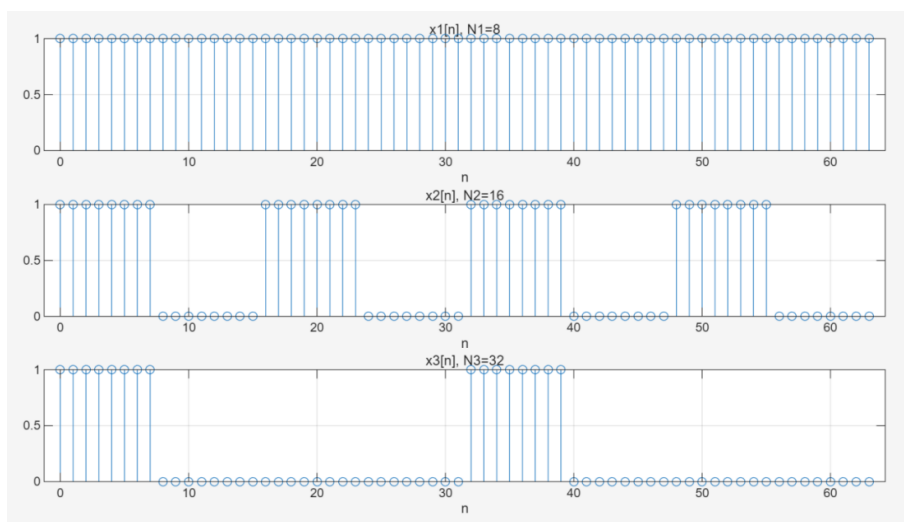


Figure 11: P3(d): Three square waves over  $0 \leq n \leq 63$ .  $x_1$  is all-ones;  $x_2$  has 50% duty cycle;  $x_3$  has 25% duty cycle.

(e) Compute DTFS coefficients with `dtfs.m` and plot  $|a_k|$

code:

```

1 clear; clc;
2 x1 = ones(1,8);
3 x2 = [ones(1,8), zeros(1,8)];
4 x3 = [ones(1,8), zeros(1,24)];
5 N1=8; N2=16; N3=32;
6
7 a1 = dtfs(x1, 0);
8 a2 = dtfs(x2, 0);

```

```
9 a3 = dtfs(x3, 0);
10
11 figure;
12 subplot(3,1,1);
13 stem(0:N1-1, abs(a1));
14 title('|a_k| of x1'); xlabel('k'); grid on;
15
16 subplot(3,1,2);
17 stem(0:N2-1, abs(a2));
18 title('|a_k| of x2'); xlabel('k'); grid on;
19
20 subplot(3,1,3);
21 stem(0:N3-1, abs(a3));
22 title('|a_k| of x3'); xlabel('k'); grid on;
23
24 % DC分量验证
25 fprintf('a1(k=0) = %.4f, 预测值 = %.4f\n', a1(1), 8/N1);
26 fprintf('a2(k=0) = %.4f, 预测值 = %.4f\n', a2(1), 8/N2);
27 fprintf('a3(k=0) = %.4f, 预测值 = %.4f\n', a3(1), 8/N3);
```

### 命令行窗口

```
a1(k=0) = 1.0000, 预测值 = 1.0000
a2(k=0) = 0.5000, 预测值 = 0.5000
a3(k=0) = 0.2500, 预测值 = 0.2500
```

Figure 12: P3(e)

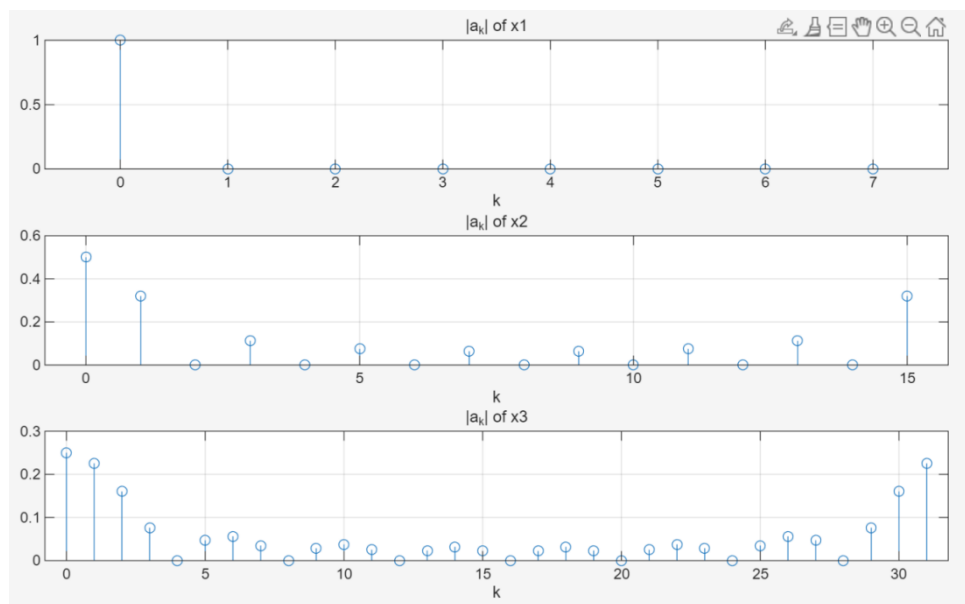


Figure 13: P3(e): Magnitude of DTFS coefficients for  $x_1$ ,  $x_2$ , and  $x_3$ .

**DC component prediction:** From the analysis equation,  $a_0 = \frac{1}{N} \sum_{n=0}^{N-1} x[n]$ , which equals the average value of one period.

- $x_1$ : all ones, so  $a_0 = 8/8 = 1.0000$  ✓
- $x_2$ : 8 ones in 16 samples, so  $a_0 = 8/16 = 0.5000$  ✓
- $x_3$ : 8 ones in 32 samples, so  $a_0 = 8/32 = 0.2500$  ✓

These match the MATLAB output exactly.

#### (f) Truncated synthesis of $x_3[n]$

code:

```

1  clear; clc;
2  x1 = ones(1,8);
3  x2 = [ones(1,8), zeros(1,8)];
4  x3 = [ones(1,8), zeros(1,24)];
5  N1=8; N2=16; N3=32;
6  n_plot = 0:31;
7  a1 = dtfs(x1, 0);
8  a2 = dtfs(x2, 0);
9  a3 = dtfs(x3, 0);
10
11 x3_2 = zeros(1,N3);

```

```
12 for k=-2:2
13     if k>=0, idx=k+1;
14     else, idx=N3+k+1; end
15     x3_2 = x3_2 + a3(idx)*exp(1j*k*(2*pi/N3)*n_plot);
16 end
17 x3_2 = real(x3_2);
18
19 x3_8 = zeros(1,N3);
20 for k=-8:8
21     if k>=0, idx=k+1;
22     else, idx=N3+k+1; end
23     x3_8 = x3_8 + a3(idx)*exp(1j*k*(2*pi/N3)*n_plot);
24 end
25 x3_8 = real(x3_8);
26
27 x3_12 = zeros(1,N3);
28 for k=-12:12
29     if k>=0, idx=k+1;
30     else, idx=N3+k+1; end
31     x3_12 = x3_12 + a3(idx)*exp(1j*k*(2*pi/N3)*n_plot);
32 end
33 x3_12 = real(x3_12);
34
35 x3_all = zeros(1,N3);
36 for k=-15:16
37     if k>=0, idx=k+1;
38     else, idx=N3+k+1; end
39     x3_all = x3_all + a3(idx)*exp(1j*k*(2*pi/N3)*n_plot);
40 end
41 x3_all = real(x3_all);
42
43 % 绘图
44 figure;
45 subplot(4,1,1); stem(n_plot, x3_2); title('x3\_2[n]: k=-2到2'); xlabel('
    n'); grid on;
46 subplot(4,1,2); stem(n_plot, x3_8); title('x3\_8[n]: k=-8到8'); xlabel('
    n'); grid on;
47 subplot(4,1,3); stem(n_plot, x3_12); title('x3\_12[n]: k=-12到12');
```

48

```

xlabel('n'); grid on;
subplot(4,1,4); stem(n_plot, x3_all); title('x3\_all[n]: 全部系数');
xlabel('n'); grid on;

```

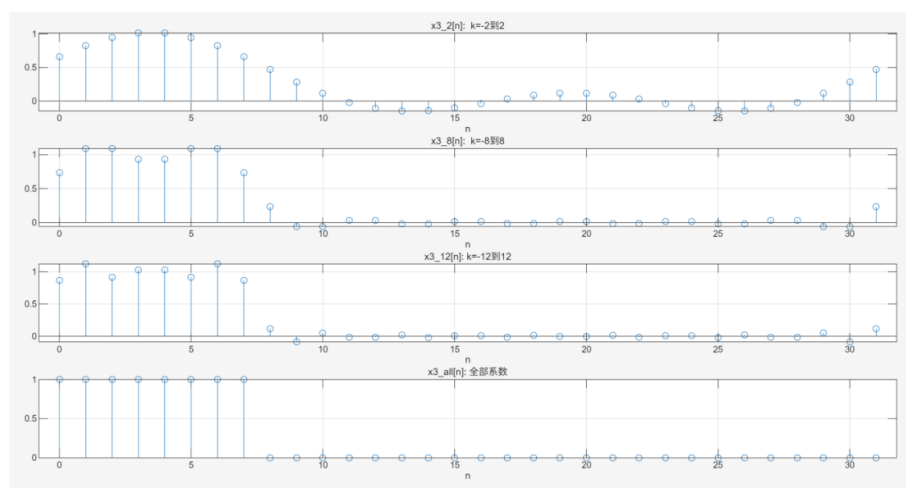


Figure 14: P3(f): Truncated DTFS synthesis of  $x_3[n]$ . As more coefficients are included, the waveform gets closer to the original square wave.

**(g) Why must  $x_{3\_all}[n]$  be a real signal?**

$x_3[n]$  is a real-valued signal, so its DTFS coefficients satisfy  $a_k = a_{-k}^*$  (conjugate symmetry). The sum  $x_{3\_all}[n] = \sum_{k=-15}^{16} a_k e^{jk(2\pi/32)n}$  uses all 32 DTFS coefficients, which is the complete DTFS of  $x_3[n]$ . By the DTFS synthesis equation, using all coefficients perfectly reconstructs the original signal  $x_3[n]$ , which is real. Therefore  $x_{3\_all}[n]$  must also be real.

We can verify this in MATLAB:

```

1 fprintf('Max error between x3_all and x3: %.2e\n', max(abs(x3_all - x3))
);

```

**Output:** Max error between x3\_all and x3: 1.78e-16

This error is at machine precision level, which confirms perfect reconstruction.

**(h) Does the synthesis show Gibbs phenomenon?**

**No, Gibbs phenomenon does not appear.**

The Gibbs phenomenon happens in continuous-time Fourier series when an infinite series is truncated: near a jump discontinuity, the partial sum always overshoots by about 9%, no matter how many terms are used.

In the discrete-time case,  $x_3[n]$  has only a finite number of DTFS coefficients ( $N_3 = 32$ ). Once all 32 coefficients are included, the signal is perfectly reconstructed with no overshoot at all. The convergence is smooth, not oscillatory. This is a key difference between continuous-time and discrete-time Fourier series.

## 4 Experimental Conclusions

- (1). When using `conv` in MATLAB, we must track the time indices manually. The output index range is  $(n_x + n_h)$  to  $(n_x + n_h) + N_x + N_h - 2$ . The overlap-add method gives exactly the same result as direct convolution, which confirms the linearity of convolution.
- (2). For 2D image filtering with separable filters, we apply 1D filters to each column and then to each row. Noncausal filtering is done by appending zeros and removing the delay. FIR filters (Remez) have linear phase and do not distort the shape of the image. The Butterworth IIR filter has nonlinear phase, which breaks image symmetry.
- (3). A real-valued periodic signal has conjugate-symmetric DTFS coefficients ( $a_k = a_{-k}^*$ ). The DC coefficient  $a_0$  equals the average value of one period. Including more DTFS coefficients in the synthesis brings the result closer to the original signal.
- (4). Discrete-time square waves do not show Gibbs phenomenon because the DTFS has only a finite number of coefficients. Perfect reconstruction is achieved when all coefficients are used.